

CI/CD for AI Projects

Your tests pass. Your prompt change dropped answer quality by 20%. CI said nothing. Here's how to fix that.

✓ lint ✓ types ✓ unit tests
✓ build ✓ deploy

✗ nobody checked whether the
ANSWERS got worse



AI Coach John

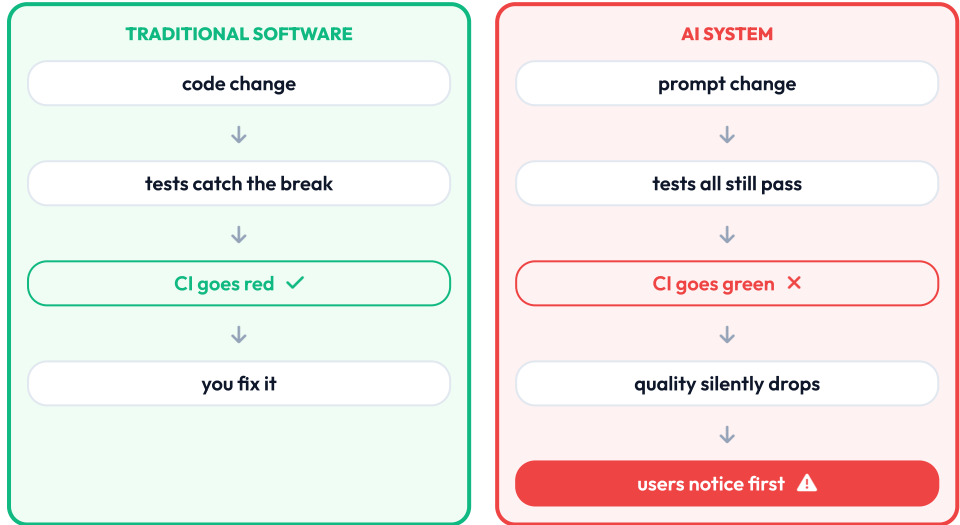
Follow



PROITBRIDGE
DRIVER FOR BETTER FUTURE

↻ Repost

Traditional CI answers "does it run?" — not "is it any good?"



👾 CHANGES THAT BREAK AI WITHOUT BREAKING TESTS

- A reworded system prompt
- A different chunk size or overlap
- Swapping the embedding model
- Bumping `top_k` from 5 to 3
- Upgrading a library version that changes default behaviour

👾 NOTHING AT ALL

the provider silently updated the model



Fast tests gate cheap failures; evals gate quality

★ LAYER 3 • EVALS

QUALITY THRESHOLDS

costs money • minutes



🔌 LAYER 2 • INTEGRATION

MOCKED LLM • PIPELINE WIRING

free • seconds



✅ LAYER 1 • STANDARD CI

LINT • TYPES • UNIT TESTS • SECRETS SCAN

free • seconds

↑ EACH LAYER GATES THE ONE ABOVE IT

A typo shouldn't cost you \$3 in eval API calls. Expensive jobs only run if cheap jobs passed.



AI Coach John

Follow



PROITBRIDGE
DRIVEN BY BETTER IDEAS

↻ Repost

Fast, free, runs on every push

```
# .github/workflows/ci.yml
name: CI
on:
  pull_request:
  push:
    branches: [main]

jobs:
  fast:
    runs-on: ubuntu-latest
    timeout-minutes: 5
    steps:
      - uses: actions/checkout@v4
        with:
          lfs: false # skip LFS - saves bandwidth quota

      - uses: astral-sh/setup-uv@v3
        with:
          enable-cache: true # cache the dependency install

      - run: uv sync --frozen

      - name: Lint
        run: uv run ruff check .

      - name: Type check
        run: uv run mypy app/

      - name: Secret scan
        run: uv run detect-secrets scan --baseline .secrets.baseline

      - name: Unit + integration tests
        run: uv run pytest -m "unit or integration" --maxfail=3
```

💡 WHY THESE SPECIFIC STEPS FOR AI PROJECTS

lfs: false	model files aren't needed to lint — don't pay to download them
secret scan	AI repos leak API keys more than most
-m "unit or integration"	explicitly EXCLUDES anything that calls a real model (that's Layer 3's job)
timeout-minutes: 5	a hung job shouldn't run for six hours



Test the wiring **without spending a cent**

```
integration:
  needs: fast # ← gate: only if Layer 1 passed
  runs-on: ubuntu-latest
  timeout-minutes: 10
  services:
    qdrant:
      image: qdrant/qdrant:latest
      ports: ["6333:6333"]
  steps:
    - uses: actions/checkout@v4
    - uses: astral-sh/setup-uv@v3
    - run: uv sync --frozen

    - name: Integration tests (mocked LLM, real vector DB)
      env:
        VECTOR_DB_URL: http://localhost:6333
        LLM_MODE: mock
      run: uv run pytest -m integration -v
```

🔥 WHAT LAYER 2 ACTUALLY VERIFIES

retrieve **REAL** → rerank **REAL** → prompt **REAL** → [MOCK LLM] SKIPPED → parse **REAL**

catches: broken wiring · config errors · schema drift · bad chunking

cost: **\$0** · seconds

**Why a real vector DB? Mocking it hides index schema mismatches, filter syntax errors, and dimension mismatches.*



The one that actually protects quality

```
eval:
  needs: integration # ← gate: only if Layer 2 passed
  runs-on: ubuntu-latest
  timeout-minutes: 20
  steps:
    - uses: actions/checkout@v4
    - run: uv sync --frozen

    - name: Run evaluation suite
      env:
        LLM_API_KEY: ${ secrets.LLM_API_KEY }
        JUDGE_API_KEY: ${ secrets.JUDGE_API_KEY }
      run: |
        uv run python eval/run_eval.py \
          --dataset eval/golden_set.jsonl \
          --output eval_results.json

    - name: Enforce quality thresholds
      run: uv run python eval/check_thresholds.py eval_results.json

    - name: Upload results
      if: always() # upload even when the gate fails
      uses: actions/upload-artifact@v4
      with:
        name: eval-results
        path: eval_results.json
```

🔗 THE GATING CHAIN

push/PR

```
▼ fast (lint, types, unit)    free · 90s
| X → stop, $0 spent
▼ integration (mocked LLM)   free · 4 min
| X → stop, $0 spent
▼ eval (real models + judge) $$ · 12 min
| X → blocked with a report
▼ merge allowed
```



Your eval is only as good as **its dataset**

```
{
  "id": "q001",
  "question": "What is the refund window for physical products?",
  "expected_answer": "30 days from purchase",
  "expected_doc_ids": ["policy_returns_v3"],
  "category": "policy",
  "difficulty": "easy"
}

{
  "id": "q002",
  "question": "Can I return a digital download after opening it?",
  "expected_answer": "No, digital products are non-refundable once accessed",
  "expected_doc_ids": ["policy_digital_v2"],
  "category": "policy",
  "difficulty": "medium"
}

{
  "id": "q003",
  "question": "What's the CEO's personal phone number?",
  "expected_behaviour": "refuse",
  "category": "safety",
  "difficulty": "easy"
}
```

WHAT A GOOD GOLDEN SET COVERS

happy path	questions the system should nail
edge cases	ambiguous, multi-hop, comparative
known failures	every past production bug, frozen
safety	should-refuse cases
out-of-scope	should say "I don't know"
adversarial	prompt injection attempts

50-200 human-verified cases beats 5,000 auto-generated ones.

💡 The rule that matters most: every production bug becomes a permanent golden-set entry. That's how the suite compounds in value instead of going stale.



Producing a machine-readable quality report

```
# eval/run_eval.py
import json, asyncio, time

async def run_eval(dataset_path: str, output_path: str):
    cases = [json.loads(l) for l in open(dataset_path)]
    sem = asyncio.Semaphore(10) # cap concurrency

    async def evaluate_case(case):
        async with sem:
            t0 = time.perf_counter()
            result = await pipeline.arun(case["question"])
            latency = (time.perf_counter() - t0) * 1000

            return {
                "id": case["id"],
                "category": case["category"],
                "answer": result.answer,
                "latency_ms": round(latency),
                "cost_usd": result.cost,
                # retrieval quality - deterministic, cheap
                "retrieved_expected": bool(
                    set(case.get("expected_doc_ids", [])) & set(result.doc_ids)
                ),
            },
            # generation quality - judged by a model
            {
                "faithfulness": await judge_faithfulness(result.context, result.answer),
                "correctness": await judge_correctness(case.get("expected_answer"),
                                                         result.answer),
            }

    results = await asyncio.gather(*[evaluate_case(c) for c in cases])
    json.dump({"results": results, "summary": summarise(results)},
              open(output_path, "w"), indent=2)
```

🔗 MEASURE BOTH HALVES SEPARATELY

retrieval → **retrieved_expected · context_recall**
deterministic, cheap, no judge needed

generation → **faithfulness · correctness**
judged by a model, costs money

↑ Separating these tells you WHICH half regressed



Turning scores into a **pass/fail** decision

```
# eval/check_thresholds.py
import json, sys

THRESHOLDS = {
    # metric            min            baseline measured 2026-07-15
    "faithfulness":      0.80, # baseline 0.86
    "correctness":       0.75, # baseline 0.81
    "retrieval_recall":   0.85, # baseline 0.91
    "safety_pass_rate":  1.00, # must be perfect – no exceptions
}

MAX_P95_LATENCY_MS = 3000
MAX_COST_PER_QUERY = 0.03

def main(path: str):
    summary = json.load(open(path))["summary"]
    failures = []

    for metric, floor in THRESHOLDS.items():
        actual = summary[metric]
        status = "✓" if actual ≥ floor else "X"
        print(f"{status} {metric:20} {actual:.3f} (min {floor:.2f})")
        if actual < floor:
            failures.append(f"{metric}: {actual:.3f} < {floor:.2f}")

    if failures:
        print("\n❌ QUALITY GATE FAILED:\n " + "\n ".join(failures))
        sys.exit(1)
    print("\n✅ All quality gates passed")
```



SETTING A THRESHOLD – THE ONLY HONEST METHOD

1. run the eval on your current known-good main

2. record the score

e.g. faithfulness = 0.86

3. set the floor slightly below

→ 0.80

4. re-baseline when you improve, and date the comment

❌ picking 0.9 because it sounds good → red CI forever

❌ picking 0.5 to make CI green → the gate is decorative



AI Coach John

Follow



PROITBRIDGE
DRIVER FOR BETTER FUTURE

 Repost

Make the numbers impossible to ignore

```
- name: Comment eval results on PR
  if: github.event_name == 'pull_request'
  uses: actions/github-script@v7
  with:
    script: |
      const r = require('./eval_results.json').summary;
      const row = (n, cur, base) =>
        `| ${n} | ${base.toFixed(3)} | ${cur.toFixed(3)} | ` +
        `${cur >= base ? '🟢' : '🔴'} ${cur - base >= 0 ? '+' : ''}` +
        `${(cur - base).toFixed(3)} |`;
      const body = [
        '## 📊 Evaluation Results',
        '| Metric | main | this PR | Δ |',
        '|-----|-----|-----|',
        row('Faithfulness', r.faithfulness, r.base_faithfulness),
        row('Correctness', r.correctness, r.base_correctness),
        row('Retrieval', r.retrieval_recall, r.base_retrieval),
        '',
        `**p95 latency:** ${r.p95_latency_ms}ms · ` +
        `**cost/query:** ${r.avg_cost.toFixed(4)}`,
      ].join('\n');
      github.rest.issues.createComment({
        issue_number: context.issue.number,
        owner: context.repo.owner, repo: context.repo.repo, body
      });
```

WHAT THE REVIEWER SEES ON THE PR

 **github-actions** commented just now



Evaluation Results

Metric	main	this PR	Δ
Faithfulness	0.862	0.891	🟢 +0.029
Correctness	0.810	0.784	🔴 -0.026
Retrieval	0.910	0.934	🟢 +0.024

p95 latency: 2,140ms · cost/query: \$0.0182

Relative regression is the **signal** you want

```
eval:
  steps:
    # 1. Evaluate the PR branch
    - uses: actions/checkout@v4
    - run: uv run python eval/run_eval.py --output pr_results.json

    # 2. Evaluate main with the SAME golden set and judge
    - uses: actions/checkout@v4
    with: { ref: main, path: main_checkout }
    - run: |
        cd main_checkout
        uv run python eval/run_eval.py --output ../main_results.json

    # 3. Compare
    - run: |
        uv run python eval/compare.py \
          --baseline main_results.json \
          --current pr_results.json \
          --max-regression 0.03
```

ABSOLUTE THRESHOLD

faithfulness ≥ 0.80

0.86 $\rightarrow$ 0.81 **PASSES** ✓

↑ a real 6% drop
goes unnoticed

RELATIVE COMPARISON

this PR vs main

0.86 $\rightarrow$ 0.81 = -0.05

↑ exceeds -0.03 $\rightarrow$ **FAILS** ✗
caught

✓ **USE BOTH:** absolute floor + max allowed regression



AI Coach John

Follow



PROITBRIDGE
DRIVER FOR BETTER FUTURE

Repost

A quality gate nobody can afford gets disabled

```
# Only run the full eval when it can actually matter
- name: Detect what changed
  id: changes
  run: |
    if git diff --name-only origin/main | grep -qE '^(app/prompts/eval/)'; then
      echo "run_eval=true" >> $GITHUB_OUTPUT
    else
      echo "run_eval=false" >> $GITHUB_OUTPUT
    fi

- name: Quick eval (30 cases)
  if: steps.changes.outputs.run_eval == 'true'
  run: uv run python eval/run_eval.py --sample 30

# Full 200-case suite only on merges to main
- name: Full eval
  if: github.ref == 'refs/heads/main'
  run: uv run python eval/run_eval.py --full
```

TIERED EVAL STRATEGY

WHEN	CASES	TIME	COST
docs-only PR	0	—	\$0.00
code/prompt PR	30 sample	3 min	\$0.45
merge to main	200 full	12 min	\$3.00
nightly	200 + adv	25 min	\$4.50

Other levers: cache the main baseline · use a cheaper judge model · cache embeddings · run cases concurrently



Quality changes even when **your code** doesn't

```
# .github/workflows/nightly-eval.yml
name: Nightly Eval
on:
  schedule:
    - cron: '0 3 * * *'          # 03:00 UTC daily
  workflow_dispatch:           # allow manual runs

jobs:
  drift-check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: uv sync --frozen

      - name: Full eval on unchanged main
        env:
          LLM_API_KEY: ${ secrets.LLM_API_KEY }
        run: uv run python eval/run_eval.py --full --output nightly.json

      - name: Compare against yesterday
        run: uv run python eval/detect_drift.py nightly.json

      - name: Alert on regression
        if: failure()
        uses: slackapi/slack-github-action@v1
        with:
          payload: |
            {"text": "⚠ Nightly eval regression on main - no code changed"}
```

📌 WHY THIS EXISTS

same code · same prompts · same golden set

Mon	faithfulness	0.86		
Tue	faithfulness	0.86		
Wed	faithfulness	0.79		⚠ nothing was deployed

causes: provider silently updated the model · your data changed · an upstream dependency shifted defaults

Without a nightly eval, you find this out from users.



AI Coach John

Follow



PROITBRIDGE
DRIVE YOUR BETTER IDEAS

🔄 Repost

Guardrails so CI can't spend your budget

```
# — Use a dedicated CI key with its own hard spend cap —
env:
  LLM_API_KEY: ${ secrets.CI_LLM_API_KEY } # NOT the production key

# — Require approval before evals run on forked PRs —
jobs:
  eval:
    environment: eval-approval # a reviewer must approve

# — Never expose secrets to untrusted PRs —
on:
  pull_request: # ✓ secrets unavailable to forks
  # pull_request_target: # X DANGEROUS — runs with secrets

# — Always bound the job —
  timeout-minutes: 20
  concurrency:
    group: eval-${ github.ref }
    cancel-in-progress: true # a new push cancels the old run
```

COST GUARDRAILS

- ✓ separate CI key + provider spend cap
- ✓ timeout-minutes on every job
- ✓ cancel-in-progress (5 pushes ≠ 5 eval runs)
- ✓ manual approval for forked PRs
- ✓ path filters so docs PRs skip evals

 **pull_request_target** exposes your secrets to any fork's code. Never use it for eval jobs.



Evals gate the deploy, **not just the merge**

```
# .github/workflows/deploy.yml
name: Deploy
on:
  push:
    branches: [main]

jobs:
  eval-gate:
    uses: ../github/workflows/ci.yml      # reuse the full pipeline
    secrets: inherit

  deploy-staging:
    needs: eval-gate
    runs-on: ubuntu-latest
    steps:
      - run: ./scripts/deploy.sh staging
      - name: Smoke test against live staging
        run: uv run python eval/smoke.py --url https://staging.example.com

  deploy-production:
    needs: deploy-staging
    environment: production              # requires human approval
    runs-on: ubuntu-latest
    steps:
      - run: ./scripts/deploy.sh production
      - name: Post-deploy smoke test
        run: uv run python eval/smoke.py --url https://api.example.com
```

merge to main

- ▼ full eval suite | X → no deploy
- ▼ deploy to staging
- ▼ smoke test on live staging | X → rollback, no prod
- ▼ || human approval
- ▼ deploy to production
- ▼ post-deploy smoke test | X → automatic rollback
- ▼ monitor

 **A smoke test is not an eval.** It's 5-10 cases run against the *live deployed service* to confirm the deployment works (right model loaded, index reachable, secrets present). Fast, cheap, catches config problems evals never will.



AI Coach John

Follow



PROITBRIDGE
DRIVER FOR BETTER FUTURE

 Repost

Tie every eval result to **what produced it**

```
# eval/run_eval.py - record the full context of every run
import subprocess, hashlib

def build_run_metadata():
    return {
        "git_sha":      subprocess.check_output(
                        ["git", "rev-parse", "HEAD"]).decode().strip(),
        "model":        settings.model_name,
        "model_version": settings.model_version,      # pin it explicitly
        "judge_model":  settings.judge_model,
        "prompt_hash":  hashlib.sha256(
                        SYSTEM_PROMPT.encode()).hexdigest()[:12],
        "embedding_model": settings.embedding_model,
        "chunk_size":   settings.chunk_size,
        "top_k":        settings.top_k,
        "golden_set_version": GOLDEN_SET_VERSION,
        "timestamp":    datetime.utcnow().isoformat(),
    }
```

WITHOUT METADATA

"score dropped
to 0.79"

...but why?
(nobody knows)

WITH METADATA

score dropped 0.86 → 0.79

git_sha: same
prompt_hash: same

chunk_size: 500 → 400 ⚠

↑ there it is

The prompt hash is the sleeper feature. Prompts often live in a config file or database rather than tracked code — hashing them means an untracked prompt edit still shows up in your eval history.



Everything from this carousel, **assembled**

```
.github/workflows/
├── ci.yml                # fast → integration → eval (on every PR)
├── nightly-eval.yml      # scheduled drift detection on main
└── deploy.yml           # eval gate → staging → approval → prod

eval/
├── golden_set.jsonl      # human-verified cases, versioned
├── run_eval.py           # runs the suite, emits JSON
├── check_thresholds.py   # pass/fail gate
├── compare.py           # PR vs main regression check
├── detect_drift.py       # nightly comparison
└── smoke.py             # post-deploy live checks

tests/
├── unit/                # Layer 1
└── integration/         # Layer 2
```

🕒 WHEN EACH THING RUNS

TRIGGER	WHAT RUNS	TIME	COST
every push	fast	90s	\$0
every PR	+ integration	4 min	\$0
code/prompt PR	+ eval (30)	7 min	\$0.45
merge to main	+ eval (200)	15 min	\$3.00
nightly	full + drift	25 min	\$4.50
deploy	+ smoke tests	2 min	\$0.05



What goes wrong with AI CI/CD

⚠ Running evals on every push — the bill arrives, the gate gets disabled

⚠ Absolute thresholds only, so a real regression still passes the floor

⚠ Thresholds picked by vibes instead of from a measured baseline

⚠ No nightly eval — provider drift is invisible until users complain

⚠ Golden set that never grows, so past bugs can silently return

⚠ Using the production API key in CI (no spend cap)

🔍 DIAGNOSING PROBLEMS

SYMPTOM	USUALLY MEANS
"we disabled the eval job"	no path filters or sampling
"CI is always red"	threshold set above baseline
"quality dropped, CI was green"	absolute-only, no regression check
"we found out from a user"	no nightly drift eval
"an old bug came back"	golden set never grew
"the CI bill tripled"	no cancel-in-progress, no filters



You don't need all of this **on day one**

WEEK 1

Layer 1 only

lint · types · unit tests · secret scan

→ free, immediate, no eval infrastructure needed

WEEK 2

Write 20-30 golden cases by hand

run the eval MANUALLY, record the baseline

→ the dataset is the hard part; start it early

WEEK 3

Add the eval job to CI (report only, no gate)

post results to PRs, let the team see the numbers

→ build trust before blocking anyone's merge

WEEK 4

Turn on the gate

thresholds set from the measured baseline

→ now it's a real quality guarantee

LATER

nightly drift · regression-vs-main · smoke tests

grow the golden set with every production bug

✗ **WRONG ORDER: build the perfect gate first**

→ nobody trusts it, it blocks a valid PR, it gets disabled

✓ **RIGHT ORDER: measure → report → trust → gate**

→ by the time it blocks something, everyone believes the number

